Computer Graphics

Graphics Pipeline

APIs

Dr. Hardik Jain
Assistant Professor
Department of Computer Science and Engineering
IIT Jodhpur

Reference slides of Dr. Bernhard Kainz

1

The Conceptual Graphics Pipeline

Modelling Transformations

Illumination (Shading)

Viewing Transformation (Perspective / Orthographic)

Clipping

Projection (to Screen Space)

Scan Conversion (Rasterization)

Visibility / Display

❑Input:
- Geometric model
- Illumination model
- Camera model
- Viewport

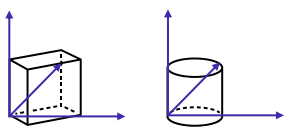
✓Output: 2D image for framebuffer display

2

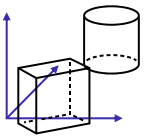
The Conceptual Graphics Pipeline

- Modelling Transformations
- Illumination (Shading)
- Viewing Transformation (Perspective / Orthographic)
- Clipping
- Projection (to Screen Space)
- Scan Conversion (Rasterization)
- Visibility / Display

- 3D models are defined in their own coordinate system
- Modeling transformations orient the models within a common coordinate frame (world coordinates)



Object Space



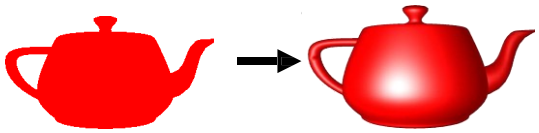
World Space

3

The Conceptual Graphics Pipeline

- Modelling Transformations
- Illumination (Shading)
- Viewing Transformation (Perspective / Orthographic)
- Clipping
- Projection (to Screen Space)
- Scan Conversion (Rasterization)
- Visibility / Display

- Vertices are lit (shaded) according to material properties, surface properties and light sources



4

The Conceptual Graphics Pipeline

- Modelling Transformations
- Illumination (Shading)
- **Viewing Transformation (Perspective / Orthographic)**
- Clipping
- Projection (to Screen Space)
- Scan Conversion (Rasterization)
- Visibility / Display

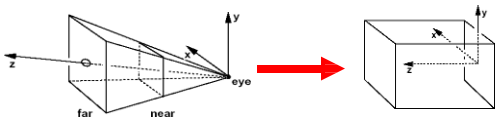
- Maps world space to eye (camera) space (matrix evaluation)
- Viewing position is transformed to origin and viewing direction is oriented along some axis (typically z)

5

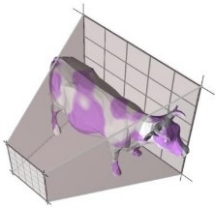
The Conceptual Graphics Pipeline

- Modelling Transformations
- Illumination (Shading)
- Viewing Transformation (Perspective / Orthographic)
- **Clipping**
- Projection (to Screen Space)
- Scan Conversion (Rasterization)
- Visibility / Display

- Portions of the scene outside the viewing volume (view frustum) are removed (clipped)
- Transform to Normalized Device Coordinates



Eye space NDC

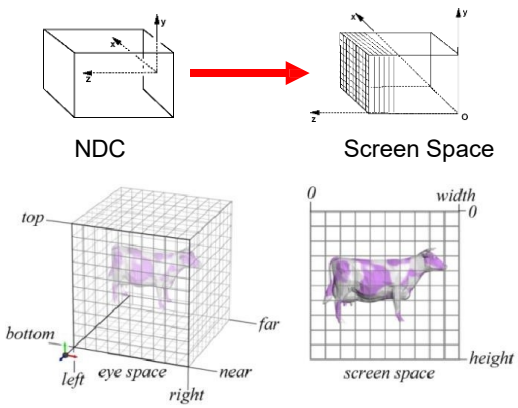


6

The Conceptual Graphics Pipeline

- Modelling Transformations
- Illumination (Shading)
- Viewing Transformation (Perspective / Orthographic)
- Clipping
- **Projection (to Screen Space)**
- Scan Conversion (Rasterization)
- Visibility / Display

- The objects are projected to the 2D imaging plane (screen space)

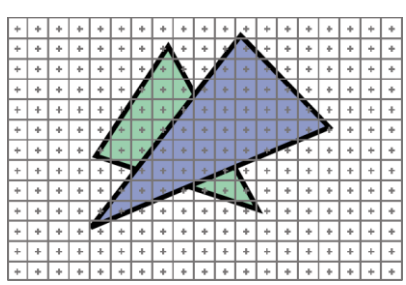


7

The Conceptual Graphics Pipeline

- Modelling Transformations
- Illumination (Shading)
- Viewing Transformation (Perspective / Orthographic)
- Clipping
- Projection (to Screen Space)
- **Scan Conversion (Rasterization)**
- Visibility / Display

- Rasterizes objects into pixels
- Interpolate values inside objects (color, depth, etc.)



8

The Conceptual Graphics Pipeline

Modelling Transformations

Illumination (Shading)

Viewing Transformation (Perspective / Orthographic)

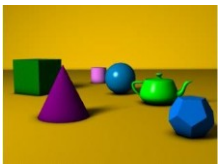
Clipping

Projection (to Screen Space)

Scan Conversion (Rasterization)

Visibility / Display

- Handles occlusions and transparency blending
- Determines which objects are closest and therefore visible
- Depth buffer



9

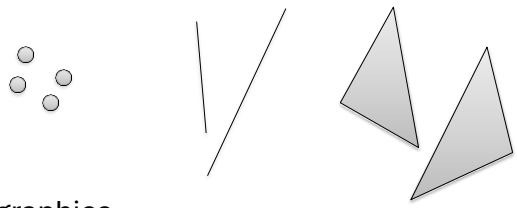
What do we want to do?

- Computer-generated imagery (CGI) in real-time
- Very computationally demanding:
  - full HD at 60hz: 1920 x 1080 x 60hz = 118 Mpx/s
  - and that's just the output data
- What could be ways to achieve this real time performance?

10

Solution

- Most of real-time graphics is based on rasterization of graphic primitives
  - Points
  - Lines
  - Triangles

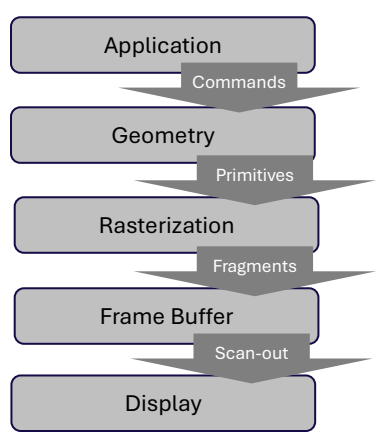


- Implemented in hardware
  - For immediate mode (real-time) graphics
  - Uses Graphics processing unit (GPU)
  - Controlled through an API such as OpenGL

The Implementation Graphics Pipeline

High-level view

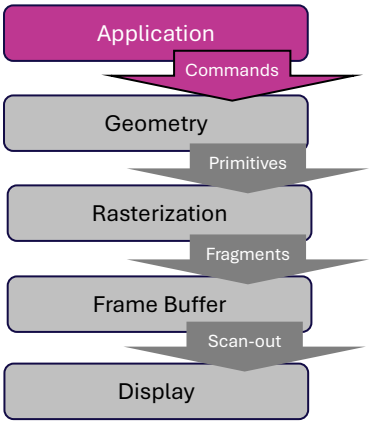
- Pipeline is split into stages which transform the data representation
- Input:
  - Geometric model
  - Illumination model
  - Camera model
  - Viewport
- Output: 2D image for framebuffer display



The Implementation Graphics Pipeline

Application Stage

- Runs on the CPU and builds the scene each frame: updates animations, handles input, manages resources, and issues draw commands (e.g., glDraw*)
- Outputs: A stream of commands plus buffers (vertices, indices, states, textures) that the GPU will consume

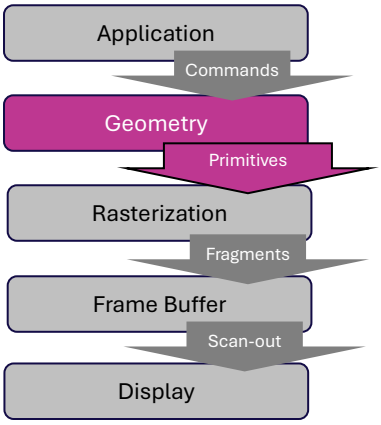


13

The Implementation Graphics Pipeline

Geometry stage

- Takes commands and vertex data and builds geometric primitives (points, lines, triangles).
- Outputs: Applied modelling and viewing transformations, projection, clipping before passing these assembled primitives

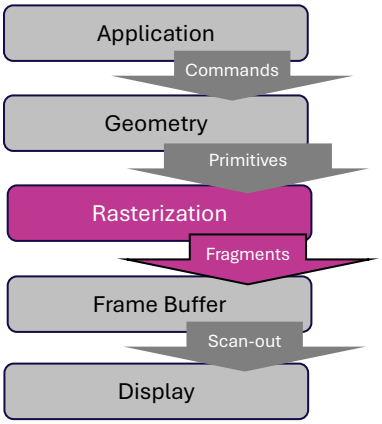


14

The Implementation Graphics Pipeline

Rasterization stage

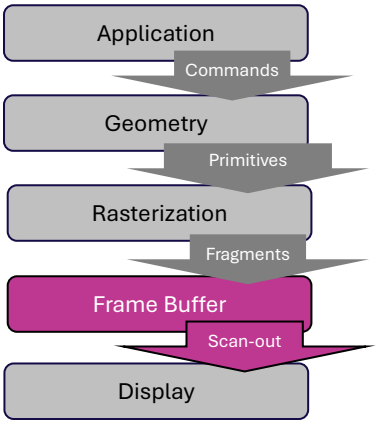
- Converts screen-space primitives into many small samples called fragments
- Outputs: These fragments by interpolating attributes (color, depth, texture coords) across each primitive for every fragment



The Hardware Graphics Pipeline

Frame buffer

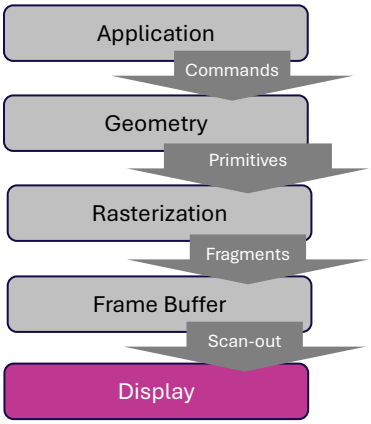
- Receives fragments and combines them into final pixels using depth tests, blending, and other per-fragment operations
- Outputs: The complete per-pixel RGBA image in memory ready to be shown on the display



The Hardware Graphics Pipeline

Display

- The display hardware periodically reads (scan-out) pixel data from the frame buffer
- Those pixels are converted into light on your monitor, producing the final visible image



Graphics Pipeline

Declarative

- Focuses on what and not How
- Describe the desired *final state* (what you want to see), letting the system figure out rendering steps
- **Pros**: More readable, less boilerplate, often better performance (framework optimization), easier state management
- **Cons**: Less direct control over low-level execution, requires the system to have pre-built logic

Imperative

- Focuses on How and not What
- Provide explicit, step-by-step *instructions* (how to draw it), offering fine-grained control but more manual management
- Objects can be drawn independent from each other
- **Pros**: High flexibility, direct control, good for complex custom tasks
- **Cons**: More boilerplate, harder to manage state, complex to optimize

Graphics APIs

Low-level 3D APIs

- **OpenGL**: Cross-platform, long-lived rasterization API that exposes a programmable pipeline (shaders in GLSL) and is widely used on desktop systems.
- **OpenGL ES**: Cut-down version of OpenGL designed for embedded and mobile devices (Android, iOS, consoles), trading some features for lower power and simpler implementations.
- **Direct3D (part of DirectX)**: Microsoft's graphics API for Windows and Xbox, tightly integrated with the Windows driver model and often the first to expose new GPU features on that platform

Continued...

Low-level 3D APIs

- **Vulkan**: Modern low-level API that gives explicit control over memory and synchronization, enabling high performance and multithreaded command recording at the cost of more complex code.
- **Metal**: Apple's low-overhead graphics and compute API for macOS, iOS, and related devices, designed to replace OpenGL with a more efficient, tightly integrated interface.

High-level / Declarative APIs

- **RenderMan**: Pixar's high-quality rendering API and shading language, used mainly for offline, film-quality photorealistic images rather than real-time graphics.
- **OpenSceneGraph**: C++ scene-graph library that sits on top of OpenGL, providing higher-level constructs like nodes, transforms, and culling so applications describe *what* to render, not every draw call.

Thank You

Why aren't there any other types of primitives like circles, ellipses?